MIT Academy of Engineering

Course: DevOps

Internal Assessment 1

DevOps Case Study and Ansible Automation

Case Study: QuickBite Food Delivery Application

Name:	Kartik Saste
Assessment:	Internal Assessment 1
Course:	DevOps
Institute:	MIT Academy of Engineering
Date:	November 2026
GitHub repository:	https://github.com/kartiksaste123/quickbite-devops-ansible
Demo video:	<Demo video link>

1. Introduction and Scenario

For this assignment, I chose a food delivery system as my real-world scenario. I order food on apps like Swiggy and Zomato quite often, and I was curious about what happens on the server side when I press "Place order". A food delivery app also fits DevOps well. It has clear peak hours at lunch and dinner, menus and offers change almost every day, and if the app is down for even half an hour, restaurants lose orders and customers just switch to another app. So the team has to release changes often but still keep the system stable, which is the problem DevOps tries to solve.

I built a small version of this kind of app and called it QuickBite. It is an order API written in Python with Flask. The `/` endpoint returns basic information about the app, `/health` checks whether the database can be reached, `/api/menu` lists menu items from three sample restaurants (Spice Garden, Pune Biryani House, and Daily Dosa), and `/api/orders` lets a customer place an order or see existing orders. I kept the application simple on purpose, because the main focus is the DevOps side: how the app is built, tested, deployed, and monitored, and how the servers are configured with Ansible.

A real food delivery platform has customers who browse menus and place orders, restaurant partners who prepare the orders, delivery partners, and an operations team that keeps everything running. My version only implements the customer side, but the server layout is the same one a bigger app would start with.

QuickBite runs on three servers. web01 runs Nginx as a reverse proxy, app01 runs the Flask app under Gunicorn as a systemd service, and db01 runs PostgreSQL 14, which stores the menu and the orders. All three also run Prometheus node_exporter on port 9100. I did not have three virtual machines, so for the lab, I used three Ubuntu 22.04 Docker containers that run systemd and an SSH server. Ansible connects to them over SSH the same way it would connect to normal VMs, so the playbook would work on real servers without changes.

2. DevOps Lifecycle for QuickBite

I divided the lifecycle into eight stages: Plan, Code, Build, Test, Release, Deploy, Operate, and Monitor. The first four are mostly on the development side, and the last four are on the operations side. Figure 1 shows the stages with the tools I selected for each one. The dashed arrow from Monitor back to Plan is important because problems seen in production should become new work items, so the cycle keeps going.

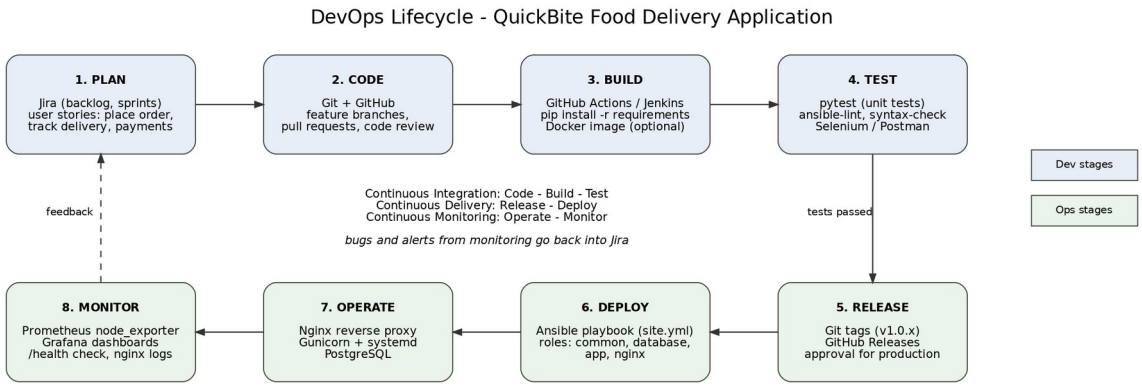


Figure 1: DevOps lifecycle of the QuickBite food delivery application

2.1 Plan

For QuickBite, the backlog would be kept in Jira as user stories, for example "customer can place an order", "customer can track the delivery", and "add online payment". Each sprint picks a few of them. I

think infrastructure work should be planned the same way. Adding a second app server before a festival sale should be a ticket like any other change.

2.2 Code

All code is kept in Git and pushed to GitHub. Every change is made in a feature branch and merged into main through a pull request after review. In my repository, the Ansible inventory, roles, playbook, and CI workflow sit next to the Python code, so the server configuration also has a history and goes through review. This is what infrastructure as code means in practice.

2.3 Build

Python has no compile step, so here the build means installing the dependencies from `requirements.txt` in a clean environment. I pinned the versions (Flask 3.0.3, Gunicorn 22.0.0, and psycopg2-binary 2.9.10) so the CI runner and the servers get the same packages. GitHub Actions does this on every push. Jenkins could do the same job, and later the app could be packaged as a Docker image, but I did not need that here.

2.4 Test

There are three types of checks. Unit tests in `tests/test_app.py` are run with pytest. They check that the index endpoint returns the app name and that `/health` returns 503 when the database is not reachable, which matters because the CI machine has no database. The automation code is checked with `ansible-playbook --syntax-check` and `ansible-lint`. For a bigger version, I would also add API tests in Postman and browser tests in Selenium.

2.5 Release

When all checks pass on main, the version is tagged in Git (for example, v1.0.x) and a GitHub Release is created. Staging can be deployed automatically, but for production I would keep a manual approval, because a broken release at dinner time directly affects orders.

2.6 Deploy

Deployment is done with the Ansible playbook `site.yml`, which uses four roles: common, database, app, and nginx. The same playbook is used for every environment, and only the inventory changes, so staging and production stay as similar as possible. It is one command and gives the same result every time, which is much safer than logging in to each server and typing commands by hand.

2.7 Operate

Here the app is serving traffic. Nginx accepts requests and passes them to Gunicorn, systemd keeps the QuickBite service running and restarts it on failure (`Restart=on-failure`), and PostgreSQL stores the data. I also used Ansible ad-hoc commands to check services on all servers at once, and the playbook to fix a configuration I broke on purpose (Section 8.6).

2.8 Monitor

Every server runs node_exporter, which exposes CPU, memory, disk, and network metrics on port 9100. In the full design, Prometheus scrapes these metrics, and Grafana shows them on dashboards. The `/health` endpoint tells whether the app can reach the database, and Nginx writes separate QuickBite access and error logs. In my lab, I installed node_exporter on all servers and checked that it returns metrics, but I did not set up the Prometheus and Grafana servers, as that was outside the scope of this assignment. Alerts and bugs go back into Jira, and the cycle starts again.

3. Development, Testing, Deployment, and Monitoring Stages

Table 1 maps the development, testing, deployment, and monitoring stages to the lifecycle above, the tools used, and where each one is in my repository.

Table 1: Main stages identified for QuickBite

Stage	What happens for QuickBite	Tools	Where in my project
Development (Plan, Code, Build)	User stories are written, code and Ansible files are changed in a feature branch, dependencies are installed	Jira, Git, GitHub, pip	roles/app/files/ quickbite/, requirements.txt, roles/
Testing	Unit tests of the API, syntax check and lint of the playbook on every push and pull request	pytest, ansible-playbook --syntax-check, ansible-lint, GitHub Actions	tests/ test_app.py, .github/workflows/ci.yml
Deployment (Release, Deploy)	Version is tagged, then the playbook configures the DB, app and web servers and a verify play checks the result	Git tags, GitHub Releases, Ansible	site.yml, roles/, inventory/
Monitoring (Operate, Monitor)	Server metrics, health check of app and database, Nginx logs, feedback to the backlog	node_exporter, Prometheus, Grafana, /health	roles/common, verify play in site.yml

4. Architecture

Figure 2 shows the architecture that the playbook creates. It is a normal three-tier setup. A customer request first reaches Nginx on web01 port 80, which is mapped to 8088 on my machine, so I use `http://127.0.0.1:8088` in the curl commands. Nginx forwards it to the upstream `quickbite_app`, which points to Gunicorn on app01, port 5000, and the Flask app connects to PostgreSQL on db01, port 5432, using psycopg2.

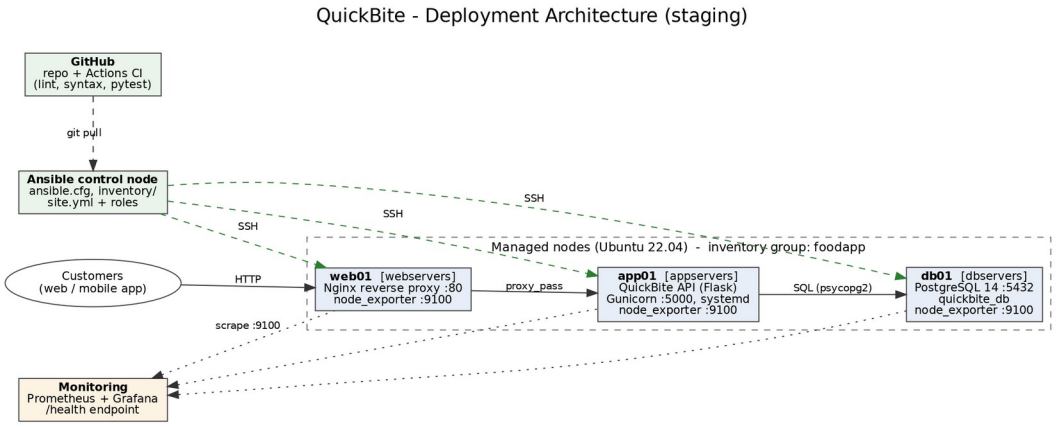


Figure 2: Deployment architecture of QuickBite (staging)

I kept the tiers on separate servers so that the app tier can be scaled by adding more app servers behind Nginx at peak hours, and so the database is never exposed to customers. The only network rule the playbook adds to `pg_hba.conf` is for the app servers to reach `quickbite_db`, with scram-sha-256 authentication.

The other part of the diagram is the management side. The Ansible control node is my Debian machine. Ansible reads the inventory and connects to each server over SSH as the user `ansible` with a key stored in `~/.ssh/quickbite/`. That user has passwordless sudo, and `become` is enabled in `ansible.cfg`. One detail that took me some time to understand is that each server has two addresses in this lab. `ansible_host` (127.0.0.1 with a different port) is only used for SSH from the control node, while the servers talk to each other on their private Docker network address, which I keep in a variable called `private_ip`.

5. DevOps Workflow and CI/CD Pipeline

Figure 3 shows the workflow for a change going from a developer to production. The developer pushes a branch and opens a pull request. GitHub Actions runs ansible-lint, the syntax check, and pytest. If something fails, the developer fixes it. If everything passes, the change is merged into main and tagged. Then the playbook is run on the staging inventory, the verify play checks `/health` and `/api/menu`, and if staging is fine, the same playbook is run with the production inventory. Monitoring watches the new version, and issues go back to the developer.

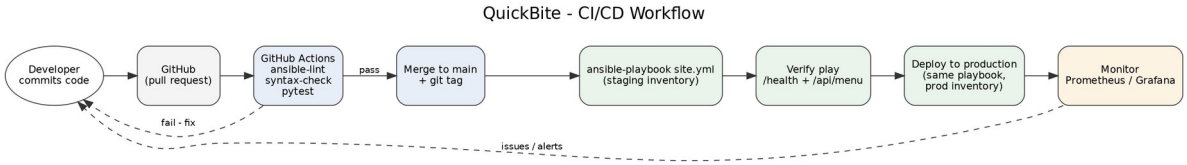


Figure 3: CI/CD workflow for QuickBite

The CI part is defined in `.github/workflows/ci.yml`, shown below.

```
name: CI

on:
  push:
    branches: [main]
  pull_request:

jobs:
  lint:
    name: Ansible lint and syntax check
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4

      - uses: actions/setup-python@v5
        with:
          python-version: "3.12"

      - name: Install ansible and ansible-lint
        run: pip install ansible ansible-lint

      - name: Install required collections
        run: ansible-galaxy collection install -r requirements.yml

      - name: Syntax check
        run: ansible-playbook site.yml --syntax-check

      - name: ansible-lint
        run: ansible-lint

  test:
    name: Unit tests for the order API
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4

      - uses: actions/setup-python@v5
        with:
```

```
          python-version: "3.12"

      - name: Install dependencies
        run: pip install -r roles/app/files/quickbite/requirements.txt pytest

      - name: Run pytest
        run: pytest -v tests/
```

The workflow runs on every push to main and on every pull request. It has two jobs, `lint` and `test`, which run in parallel on a fresh `ubuntu-latest` runner with Python 3.12. The lint job installs Ansible, ansible-lint, and the collections from `requirements.yml` (community.general and community.postgresql), then runs the syntax check and ansible-lint. The test job installs the app requirements and pytest, then runs the unit tests. A pull request should only be merged when both jobs pass.

The pipeline does not deploy by itself yet, because my lab servers are on my own machine and GitHub cannot reach them. I run the deploy step from the control node after the checks pass. In a real setup, I would add a third job that runs on tags and calls `ansible-playbook` with the production inventory from a self-hosted runner, with the SSH key stored as a GitHub secret.

6. Ansible Inventory and Configuration

The file `ansible.cfg` sets the defaults for the project. It points to the inventory and roles folders, turns off host key checking (the lab containers are rebuilt often, so their host keys can change), switches on sudo with `become`, and enables SSH pipelining to make runs faster.

```
[defaults]
inventory = inventory/hosts.ini
roles_path = roles
host_key_checking = False
interpreter_python = /usr/bin/python3
retry_files_enabled = False
forks = 10

[privilege_escalation]
become = True
become_method = sudo

[ssh_connection]
pipelining = True
```

6.1 Hosts and groups

The inventory is in INI format in `inventory/hosts.ini`. I made one group per tier and a parent group for the whole application.

```
# QuickBite food delivery - staging inventory
# The 3 servers are Ubuntu 22.04 containers started by lab/start_lab.sh,
# each one exposes ssh on a different port of the control node.

[webservers]
web01 ansible_host=127.0.0.1 ansible_port=2221 http_port_public=8088

[appservers]
app01 ansible_host=127.0.0.1 ansible_port=2222

[dbservers]
db01 ansible_host=127.0.0.1 ansible_port=2223

# parent group with every server of the application
[foodapp:children]
webservers
appservers
dbservers

[foodapp:vars]
env_name=staging
```

There are three groups: `webservers`, `appservers`, and `dbservers`, with one host each. All three containers are on my machine, so they share `ansible_host=127.0.0.1` with different `ansible_port` values (2221, 2222, 2223). web01 also has `http_port_public=8088`, which the verify play uses to call the app from outside. The group `foodapp` is made with `:children`, so it contains all three tier groups, and I can target the whole application with `hosts: foodapp`. `[foodapp:vars]` sets `env_name=staging`. For production, I would add a second inventory with the same groups, different hosts, and `env_name=production`.

6.2 Group variables and host variables

I kept most variables in `group_vars` and `host_vars` because YAML is easier to read. `group_vars/all.yml` applies to every host and has the SSH user and key, the common packages, the ports of each tier, and the database details. The database values are here and not under dbservers because both the app and database roles need them.

```
---
# common settings for every host
project_name: quickbite
timezone: Asia/Kolkata


ansible_user: ansible
ansible_ssh_private_key_file: ~/.ssh/quickbite/ansible_key


common_packages:
  - curl
  - vim
  - htop
  - net-tools
  - tzdata
  - prometheus-node-exporter


# ports used by the different tiers
app_port: 5000
db_port: 5432
node_exporter_port: 9100


# database details (shared by the app and db roles)
db_name: quickbite_db
db_user: quickbite
# lab password only - in a real setup this goes into ansible-vault
db_password: "Qb_lab_2026"


# servers talk to each other on their private (docker bridge) address,
# ansible_host is only the ssh endpoint used by the control node
private_ip: "{{ ansible_default_ipv4.address }}"
```

Each tier group has its own small file, and each host has a file in `host_vars`:

```
==> inventory/group_vars/appservers.yml <==
---
app_user: quickbite
app_dir: /opt/quickbite
app_venv: /opt/quickbite/venv
app_config_dir: /etc/quickbite
gunicorn_workers: 2
flask_env: staging


==> inventory/group_vars/dbservers.yml <==
---
postgres_version: 14
postgres_listen_addresses: "*"
postgres_max_connections: 100


==> inventory/group_vars/webservers.yml <==
---
nginx_server_name: quickbite.local
nginx_listen_port: 80
nginx_worker_connections: 1024
nginx_client_max_body_size: 2m


==> inventory/host_vars/app01.yml <==
---
server_role: "Application server - QuickBite order API (Flask + Gunicorn)"
```

```
gunicorn_workers: 3

==> inventory/host_vars/db01.yml <==
---
server_role: "Database server - PostgreSQL"
postgres_max_connections: 150

==> inventory/host_vars/web01.yml <==
---
server_role: "Web server - Nginx reverse proxy"
```

The host variables show how precedence works. `gunicorn_workers` is 2 in `group_vars/appservers.yml` but 3 in `host_vars/app01.yml`, and `postgres_max_connections` is 100 for the group and 150 for db01. Host variables win over group variables, which I confirmed with `ansible-inventory --host app01 --yaml` (Figure 4). Each host also has a `server_role` text that the common role prints in the login banner.

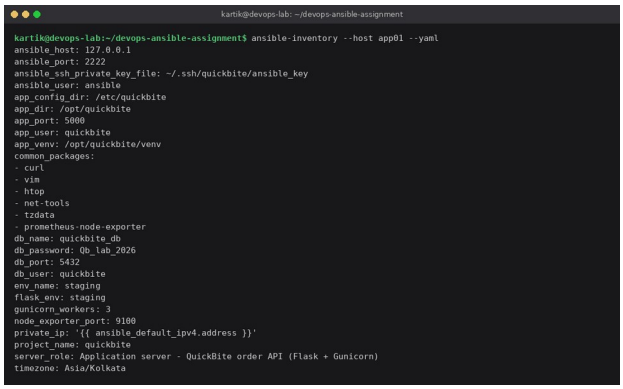


Figure 4: Merged variables for app01 (ansible-inventory --host app01 --yaml)

`private_ip` is filled from the gathered facts, so I never type private IP addresses. The Nginx upstream and the pg_hba.conf rules loop over `groups['appservers']` and read the `private_ip` of each host. So adding a second app server only needs one new line under `[appservers]`. On the next run, Nginx balances between both app servers, and PostgreSQL accepts the new one.

7. Ansible Playbook and Roles

7.1 Main playbook (site.yml)

`site.yml` is the main playbook. It is short because all the real work is inside the roles. It has five plays, and each play targets one group.

```
---
# QuickBite food delivery - main playbook
# usage: ansible-playbook site.yml   (or with --tags nginx / app / database / verify)

- name: Common setup for all servers
  hosts: foodapp
  roles:
    - role: common
      tags: common

- name: Configure database server
  hosts: dbservers
  roles:
    - role: database
      tags: database

- name: Deploy QuickBite application
  hosts: appservers
  roles:
    - role: app
      tags: app

- name: Configure nginx reverse proxy
  hosts: webservers
```

```
  roles:
    - role: nginx
      tags: nginx


- name: Verify the deployment
  hosts: webservers
  become: false
  tags: verify
  tasks:
    - name: Check health endpoint through nginx
      ansible.builtin.uri:
        url: "http://{{ ansible_host }}:{{ http_port_public }}/health"
        return_content: true
      register: health
      delegate_to: localhost
      until: health.status == 200
      retries: 5
      delay: 3


    - name: Get menu through nginx
      ansible.builtin.uri:
        url: "http://{{ ansible_host }}:{{ http_port_public }}/api/menu"
        return_content: true
      register: menu
      delegate_to: localhost


    - name: Show app response
      ansible.builtin.debug:
        msg:
          - "Health : {{ health.json }}"
          - "Menu items returned: {{ menu.json | length }}"
          - "First item: {{ menu.json[0] }}"
```

The order of the plays matters. The common role runs first on all hosts. The database comes before the app, because when Gunicorn loads the app, it creates the tables and inserts the menu, so the database must already exist. Nginx comes after the app, and the last play verifies the whole chain. It runs the uri module from the control node (`delegate_to: localhost`) and calls `/health` and `/api/menu` through Nginx like a customer would, retrying up to five times because Gunicorn may take a moment to start. Each role has a tag, so one part can be run alone, for example with `--tags nginx,verify`.

7.2 Roles

All four roles use the standard layout (tasks, handlers, templates, files, defaults). Values a user may want to change are in `defaults/main.yml` or the inventory, not inside tasks, so a role like nginx or database can be reused in another project by only changing variables.

The common role updates the apt cache (`cache_valid_time: 3600`, so not on every run), installs the common packages (including prometheus-node-exporter), sets the timezone to Asia/Kolkata, starts node_exporter, and writes a login banner from `motd.j2`, which warns that the server is managed by Ansible.

The database role installs PostgreSQL, edits `postgresql.conf` with lineinfile (listen address, port, and max_connections), adds a pg_hba rule for every app server, and creates the database user and database with the community.postgresql modules. This part was the most interesting for me:

```
- name: Allow app servers to connect (pg_hba.conf)
  community.postgresql.postgresql_pg_hba:
    dest: "{{ database_conf_dir }}/pg_hba.conf"
    contype: host
    databases: "{{ db_name }}"
    users: "{{ db_user }}"
    source: "{{ hostvars[item].private_ip }}/32"
    method: scram-sha-256
  loop: "{{ groups['appservers'] }}"
  notify: Restart postgresql


# restart now (if needed) so the app can connect in the next play
- name: Flush handlers
  ansible.builtin.meta: flush_handlers
```

Handlers normally run at the end of a play, but PostgreSQL has to restart with the new settings before the app play starts, so I added `meta: flush_handlers`. The task that creates the user has `no_log: true`, so the password is not printed.

The app role installs Python, creates a system user `quickbite` with no login shell, copies the code to `/opt/quickbite`, creates a virtualenv with the requirements, writes the environment file and the systemd unit from templates, starts the service, and waits for port 5000. The unit template:

```
# managed by ansible
[Unit]
Description=QuickBite food delivery API
After=network.target


[Service]
User={{ app_user }}
Group={{ app_user }}
WorkingDirectory={{ app_dir }}
EnvironmentFile={{ app_config_dir }}/quickbite.env
ExecStart={{ app_venv }}/bin/gunicorn --workers {{ gunicorn_workers }} --bind 0.0.0.0:{{ app_port }}
--preload app:app
Restart=on-failure
RestartSec=3


[Install]
WantedBy=multi-user.target
```

The nginx role installs Nginx, sets `worker_connections`, deploys the site from a template, enables it with a symlink, removes the default site, tests the config with `nginx -t`, and makes sure Nginx is running. The upstream block is built from the inventory:

```
upstream quickbite_app {
{% for host in groups['appservers'] %}
```

7.3 Handlers, templates, and idempotency

Each role that manages a service has handlers: "Reload systemd" and "Restart quickbite" in the app role, "Restart postgresql" in the database role, and "Reload nginx" in the nginx role. Tasks that change a config file notify them, so a service is restarted only when its configuration really changes. For Nginx, I used reload instead of restart so open connections are not dropped. Jinja2 templates are used for the Nginx site, the app environment file, the systemd unit, and the banner, and all values in them come from the inventory, so there are no hard-coded IPs or passwords in the roles.

I tried to make the playbook fully idempotent, meaning a second run changes nothing if the servers are already correct. Modules like apt, user, file, template, and service describe a state and not an action, so they are idempotent already. The only command task is `nginx -t`, which only checks the config, so I marked it `changed_when: false`. I also used full module names (for example, `ansible.builtin.apt`) and named every task, as the ansible-lint production profile expects.

8. Execution and Output

All outputs here are from my own runs with ansible-core 2.21.6 and Python 3.13.5 in a virtual environment. The full text of every command is in the `evidence/` folder, and the screenshots are in `screenshots/`.

8.1 Lab servers, connection, and inventory

First, I started the servers with `./lab/start_lab.sh`, which creates an SSH key, builds the image, and starts web01, app01, and db01 with the port mappings (Figure 5). Then I checked the connection with the ping module (Figure 6) and the group structure with `ansible-inventory --graph` (Figure 7).

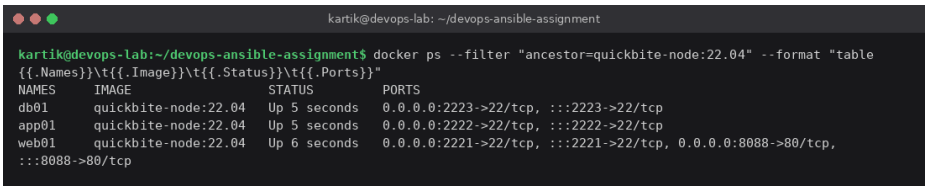


Figure 5: The three lab servers running (docker ps)

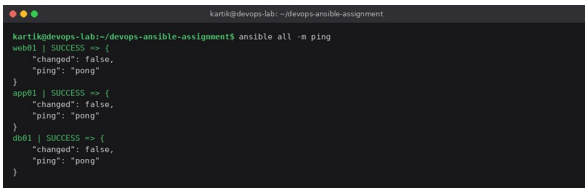


Figure 6: ansible all -m ping



Figure 7: ansible-inventory --graph

8.2 Syntax check, lint, and unit tests

Before the real run, I ran the same checks as CI. The syntax check passed, ansible-lint reported 0 failures and 0 warnings in 31 files with the production profile (the strictest one), and both pytest tests passed (Figure 8).

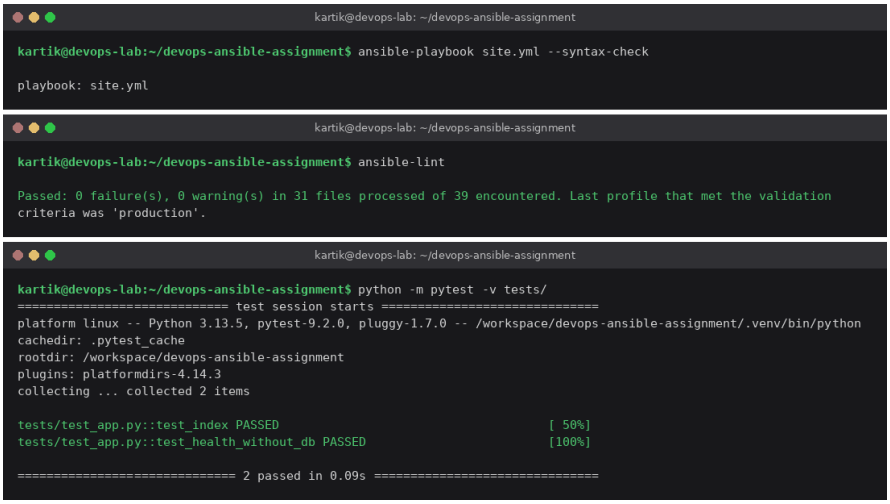


Figure 8: Syntax check, ansible-lint (production profile) and pytest (2 passed)

8.3 First run of the playbook

On the first run (`ansible-playbook site.yml`), almost every task shows changed, because the servers were empty containers. Figure 9 shows the end of the run. The verify play reports the database as up, 5 menu items, and Paneer Butter Masala from Spice Garden as the first item. The PLAY RECAP shows app01 ok=18 changed=13, db01 ok=14 changed=11, and web01 ok=19 changed=11, with failed=0 and unreachable=0 everywhere. The full run is in `evidence/08_playbook_first_run.txt`.

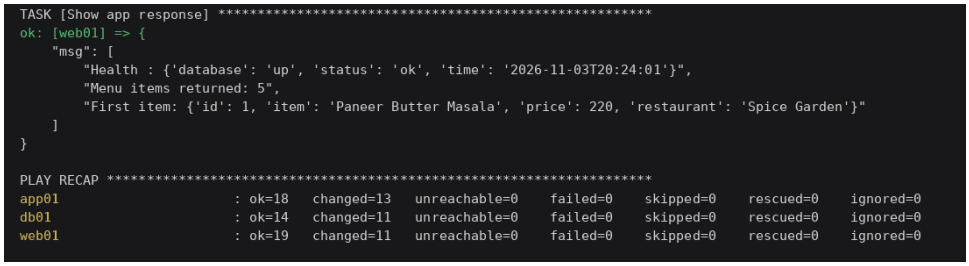


Figure 9: End of the first playbook run with the PLAY RECAP

8.4 Second run (idempotency)

Then I ran the same command again without changing anything, and every host shows `changed=0` (Figure 10). The ok counts are a bit lower (16, 13, and 18) because no handlers ran this time. app01 has two handlers, while db01 and web01 have one each. This means the playbook can be run any number of times safely.

```
TASK [Show app response] ********************************************************
ok: [web01] => {
    "msg": [
        "Health : {'database': 'up', 'status': 'ok', 'time': '2026-11-03T20:24:34'}",
        "Menu items returned: 5",
        "First item: {'id': 1, 'item': 'Paneer Butter Masala', 'price': 220, 'restaurant': 'Spice Garden'}"
    ]
}

PLAY RECAP **********************************************************************
app01                      : ok=16   changed=0    unreachable=0    failed=0    skipped=0    rescued=0    ignored=0
db01                       : ok=13   changed=0    unreachable=0    failed=0    skipped=0    rescued=0    ignored=0
web01                      : ok=18   changed=0    unreachable=0    failed=0    skipped=0    rescued=0    ignored=0
```

Figure 10: Second run of the same playbook, changed=0 on all hosts

8.5 Testing the app through Nginx and checking services

Next, I tested the app with curl through Nginx on port 8088 (Figure 11). The root endpoint shows `served_by: app01` and `env: staging`, so Nginx is forwarding to the app server and the templated environment file is used. `/health` reports the database as up, `/api/menu` returns five items from PostgreSQL, and a POST to `/api/orders` returned order_id 2 with status PLACED (order 1 was from an earlier test). The header shows `Server: nginx/1.18.0 (Ubuntu)`. I also noticed the time in `/health` is 20:25:46 while the Date header says 14:55:46 GMT, which is the same moment in IST, so the timezone set by the common role works.

```
kartik@devops-lab: ~/devops-ansible-assignment

kartik@devops-lab:~/devops-ansible-assignment$ curl -s http://127.0.0.1:8088/; echo
{"app":"QuickBite Food Delivery API","env":"staging","served_by":"app01"}

kartik@devops-lab:~/devops-ansible-assignment$ curl -s http://127.0.0.1:8088/health; echo
{"database":"up","status":"ok","time":"2026-11-03T20:25:46"}

kartik@devops-lab:~/devops-ansible-assignment$ curl -s http://127.0.0.1:8088/api/menu | python3 -m json.tool
[
    {
        "id": 1,
        "item": "Paneer Butter Masala",
        "price": 220,
        "restaurant": "Spice Garden"
    },
    {
        "id": 2,
        "item": "Butter Naan",
        "price": 40,
        "restaurant": "Spice Garden"
    },
    {
        "id": 3,
        "item": "Chicken Biryani",
        "price": 260,
        "restaurant": "Pune Biryani House"
    },
    {
        "id": 4,
        "item": "Veg Biryani",
        "price": 190,
        "restaurant": "Pune Biryani House"
    },
    {
        "id": 5,
        "item": "Masala Dosa",
        "price": 90,
        "restaurant": "Daily Dosa"
    }
]
kartik@devops-lab:~/devops-ansible-assignment$ curl -s -X POST -H 'Content-Type: application/json' -d '{"customer":
"Kartik", "item_id": 3, "qty": 2}' http://127.0.0.1:8088/api/orders; echo
{"order_id":2,"status":"PLACED"}

kartik@devops-lab:~/devops-ansible-assignment$ curl -s http://127.0.0.1:8088/api/orders; echo
[{"customer":"Kartik","item":"Chicken
Biryani","order_id":1,"qty":2,"status":"PLACED"},{"customer":"Kartik","item":"Chicken
Biryani","order_id":2,"qty":2,"status":"PLACED"}]

kartik@devops-lab:~/devops-ansible-assignment$ curl -sI http://127.0.0.1:8088/health
HTTP/1.1 200 OK
Server: nginx/1.18.0 (Ubuntu)
Date: Tue, 03 Nov 2026 14:55:46 GMT
Content-Type: application/json
Content-Length: 61
Connection: keep-alive
```

Figure 11: curl requests to QuickBite through the Nginx reverse proxy

With ad-hoc commands, I checked every tier (Figure 12). The QuickBite, Nginx, and PostgreSQL services are all active, and node_exporter returns over a thousand `node_` metric lines on each server (1047 on db01, 1031 on web01 and app01).

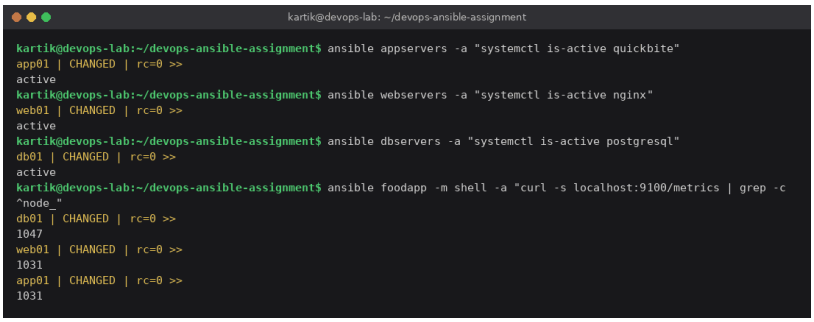


Figure 12: Service status and node_exporter metrics on all servers

8.6 Fixing configuration drift

Finally, I wanted to show why configuration management is useful after the first deployment. I removed the QuickBite site from Nginx by hand with an ad-hoc command, like a wrong manual change on a server, and then ran only the nginx and verify parts with `--tags nginx,verify` (Figure 13).

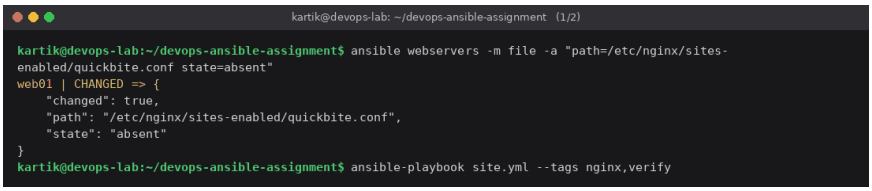


Figure 13: Removing the Nginx site by hand to simulate drift

Ansible found the missing symlink, created it again, and the handler reloaded Nginx. The verify play confirmed that the app was reachable again. The recap shows web01 ok=14 changed=2, and app01 and db01 were not touched apart from fact gathering (Figure 14).

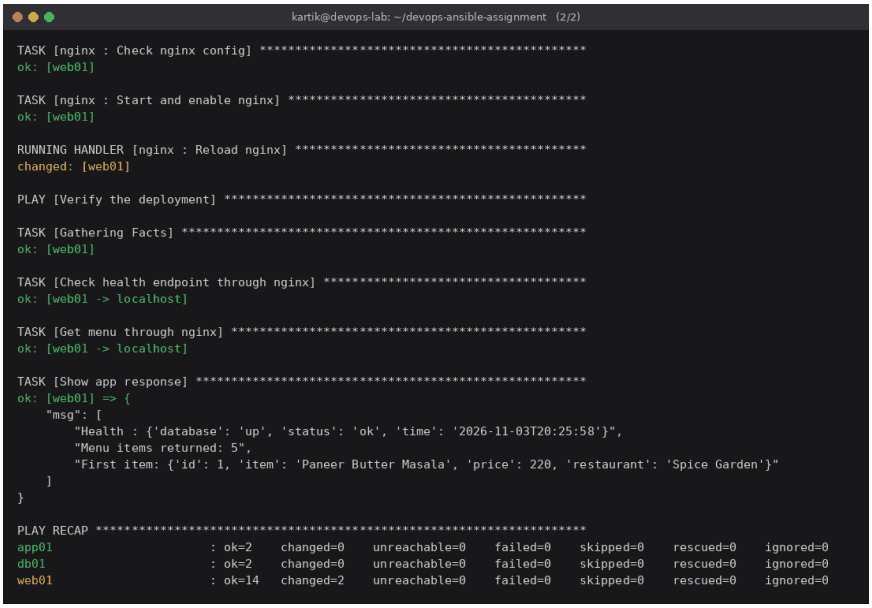


Figure 14: Playbook run with tags restores the site, web01 changed=2

9. Challenges Faced and Solutions

The first issue was networking. At first, I tried to reach the containers directly on their Docker bridge IP addresses, but because of the network and firewall setup on my host, this was not reliable. So I mapped the SSH port of each container to a port on localhost (2221, 2222, 2223) and port 80 of web01 to 8088. That fixed SSH but created a second problem. Every host now had `ansible_host=127.0.0.1`, which I could not use in the Nginx upstream or pg_hba.conf, because inside a container 127.0.0.1 is the

container itself. I solved it with the `private_ip` variable taken from facts, so SSH uses `ansible_host` and the servers use `private_ip`.

The second issue was the PostgreSQL driver. The normal `psycopg2` package from pip is built from source, so the server needs a compiler and the PostgreSQL development headers. I did not want build tools on the app server, so I used `psycopg2-binary` pinned to 2.9.10. The community.postgresql modules also need psycopg2 on the database server, and there I installed the Ubuntu package `python3-psycopg2` with apt instead.

The third point is the database password, which is in plain text in `group_vars/all.yml`. I left it like that only because this is a lab, with a comment in the file, and used `no_log: true` on the task that sets it. In a real project, I would encrypt it with Ansible Vault (`ansible-vault encrypt_string` or a separate vault file) and run the playbook with `--ask-vault-pass`, with the vault password stored as a GitHub secret for CI.

I also saw two warnings that did not cause failures. One is a deprecation warning about facts as top-level `ansible_` variables, which points to my `private_ip` line and can be fixed by using `ansible_facts['default_ipv4']['address']`. The other is a remote_tmp warning on db01, for example, when a task uses `become_user: postgres`. I plan to clean up both in the next version.

10. Conclusion

In this assignment, I took a food delivery app as a real-world scenario, planned its full DevOps lifecycle, identified the development, testing, deployment, and monitoring stages, and designed a CI/CD workflow, part of which already runs in GitHub Actions. I prepared an inventory with tier groups, a parent group, and both group and host variables, and wrote a playbook with four reusable roles that sets up PostgreSQL, the Flask app with Gunicorn, and Nginx on three servers with one command.

The playbook passed ansible-lint with the production profile, ran without failures, gave changed=0 on the second run, and fixed a manual change by itself. What I learned most is that writing the playbook was only half the job. The other half was making it safe to run again and again, which is what makes a team trust automation. Next, I would add Ansible Vault, a production inventory, an automatic deploy job, and real Prometheus and Grafana servers.

GitHub repository: https://github.com/kartiksaste123/quickbite-devops-ansible

Demo video: <Demo video link>

References

[1] Ansible Documentation, "How to build your inventory". https://docs.ansible.com/projects/ansible/latest/inventory_guide/intro_inventory.html

[2] Ansible Documentation, "Ansible playbooks". https://docs.ansible.com/projects/ansible/latest/playbook_guide/playbooks_intro.html

[3] Ansible Documentation, "Roles". https://docs.ansible.com/projects/ansible/latest/playbook_guide/playbooks_reuse_roles.html

[4] Ansible Documentation, "Handlers: running operations on change". https://docs.ansible.com/projects/ansible/latest/playbook_guide/playbooks_handlers.html

[5] Ansible Documentation, "Protecting sensitive data with Ansible vault". https://docs.ansible.com/projects/ansible/latest/vault_guide/index.html

[6] Ansible Lint Documentation. https://docs.ansible.com/projects/lint/

[7] Nginx Documentation, "Module ngx_http_proxy_module". https://nginx.org/en/docs/http/ngx_http_proxy_module.html

[8] PostgreSQL 14 Documentation, "The pg_hba.conf File". https://www.postgresql.org/docs/14/auth-pg-hba-conf.html

[9] Flask Documentation. https://flask.palletsprojects.com/en/stable/ and Gunicorn. https://gunicorn.org/

[10] GitHub Actions Documentation. https://docs.github.com/en/actions

[11] Prometheus, Node Exporter guide. https://prometheus.io/docs/guides/node-exporter/